TP 18 : Compression d'images

Ce TP est à réaliser en C.

Préliminaires

Dans ce TP, on cherche à compresser des images en restreignant le nombre de couleurs de celles-ci. En effet, une image est habituellement représentée par un ensemble de pixels, chacun ayant une valeur parmi 256^3 possibles. Il faut donc au moins $3\log_2(256) = 24$ bits pour encoder la couleur de chaque pixel sans perte. En limitant la palette de couleurs à une taille de k, il faudrait alors $\log_2(k)$ bits pour encoder la couleur de chaque pixel. Pour cela, on se propose d'utiliser l'algorithme des k-moyennes.

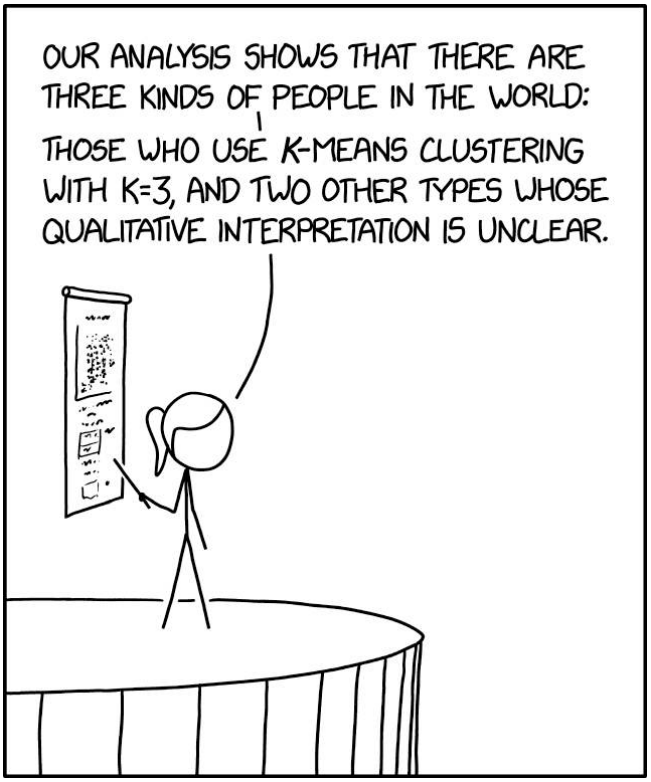

FIGURE 1 – Expliquer a posteriori le sens des groupes obtenus par un algorithme de partitionnement n'est pas toujours simple. Source : https ://xkcd.com/2731/

Nous nous intéresserons uniquement à la partie utilisant l'algorithme des k-moyennes. Le fichier généré ne sera donc pas réellement compressé mais utilisera seulement moins de couleurs.

Algorithme des k moyennes

On représente un pixel par un triplet (r, v, b) correspondant aux trois composantes colorées nécessaires à son affichage. On propose donc le type suivant :

```
struct pixel{
    int rouge;
    int vert;
    int bleu;
};

typedef struct pixel pixel_t;
```

De plus, un image peut être vue comme un tableau de pixels de dimension sa longueur fois sa hauteur. On utilisera donc le type :

```
struct image{
    int hauteur;
    int longueur;
    pixel_t* tableau;
};

typedef struct image image_t;
```

On aurait pu manipuler un tableau de tableaux de pixels mais ce n'est pas le choix fait ici pour réduire le nombre de types structurés nécessaires à l'implémentation de l'algorithme. L'image est lue ligne par ligne.

Pour la suite, on représente un ensemble de représentants de pixels comme un tableau de pixels.

On propose d'utiliser la distance euclidienne pour calculer la proximité entre deux pixels.

Question 1 : Écrire une fonction `float distance_euclide(pixel_t p1, pixel_t p2)` qui, étant donnés deux pixels, calcule leur distance.

Question 2 : Écrire une fonction `int represente_par(pixel_t* representants, int k, pixel_t p)` qui, étant donnés un tableau de pixels de taille k et un pixel, renvoie l'indice du pixel le plus proche parmi les représentants.

Question 3 : Écrire une fonction `int* represente(image_t img, pixel_t* representants, int k)` qui, étant donnés une image et un tableau de représentants, renvoie le tableau des indices des représentants. Ainsi, dans la i-ème case du tableau, on trouvera l'indice du représentant du i-ième pixel de l'image.

On propose d'initialiser les k représentants en tirant de façon aléatoire leur trois composantes.

Question 4 : Écrire une fonction `pixel_t* initialisation(image_t img, int k)` qui, étant donnés une image et un entier k, renvoie une sélection aléatoire de k pixels. On pourra utiliser `rand` qui renvoie un nombre aléatoire entre 0 et `RAND_MAX`. l'algorithme de sélection n'aura pas besoin d'être optimisé.

Question 5 : Écrire une fonction `pixel_t barycentre(image_t img, int* representes, int i)` qui, étant donnés une image, un tableau des indices des représentants respectifs et un indice i, renvoie le barycentre de l'ensemble des pixels ayant pour représentant l'élément d'indice i. Si aucun élément n'est représenté par l'élément d'indice i, on renverra un pixel dont chacune des composantes vaudra -1.

Question 6 : Écrire une fonction `pixel_t* nouvelle_etape(image_t img, int k, pixel_t* representants)` qui, étant donné une image et un tableau de représentants, renvoie le tableau des représentants obtenus après une itération de l'algorithme des k moyennes. Si un représentant ne représente aucun pixel, on ne modifiera pas sa valeur.

Manipuler des images

Dans cette partie, on rappelle les fonctions de manipulation de fichiers suivantes :

- `FILE* fopen(char* nom_fichier, char* mode)`. L'option `w` permet d'écrire dans un fichier tandis que l'option `r` permet de le lire.
- `int flcose(FILE* pointeur_fichier)`.
- `int fprintf(FILE* pointeur_fichier, char* format, ...)` qui s'utilise de façon similaire à `printf`.

- `int fscanf(FILE *pointeur_fichier, const char *format, ...)` qui s'utilise de façon similaire à `scanf`. On lui fournira donc l'adresse où écrire la valeur lue.

Pour plus de simplicité, on manipule des fichiers en format PPM (Portable PixMap file format). Le fichier correspondant à une image commence par le nombre magique P3 puis par la largeur et la hauteur de l'image. La valeur maximale d'une couleur est l'entier suivant (ici toujours 255). Chaque pixel est alors encodé par un triplet correspondant aux trois composantes.

FIGURE 2 – Un exemple très simple de fichier en format PPM.

Le fichier de la figure 2 pourra être généré de la façon suivante :

```
void main()
{
  FILE * out;
  out = fopen("color_test.ppm", "w");
  fprintf(out, "P3 256 256 255\n");
  for(int r=0; r<256; r++) {
    for(int b=0; b<256; b++) {
      fprintf(out, "%d %d %d\n", r, 0, b);
    }
  }
  fclose(out);
}
```

Question 7 : Écrire une fonction `image_t creer(int hauteur, int longueur)` qui, étant alloue l'espace nécessaire pour stocker une image de la dimension souhaitée et renvoie l'objet de type `image_t` associé.

Je vous fournis une fonction dans le fichier `main.c` une fonction `image_t initialiser(char* nom)` qui, étant donné le nom d'une image, l'ouvre et renvoie un objet de type `image_t` correspondant.

Question 8 : Écrire une fonction `void dessiner(char* destination, image_t img, pixel_t* representants , int* representes)` qui, étant donné un nom de fichier, une image d'origine, un tableau des représentants et un tableau d'indices des représentants de chacun des pixels, écrit dans le fichier l'image obtenue par compression au format PPM.

Question 9 : Écrire une fonction `void compresser(image_t img, int k, char* destination)` qui, étant donnés une image et un nom de fichier, écrit dans le fichier correspondant l'image obtenu par l'algorithme des k moyennes.

On cherche à étudier la vitesse de convergence de l'algorithme des k-moyennes.

Question 10 : Écrire une fonction `void variations(char* nom_image, int k)` qui, étant donné un nom de fichier, crée une image par itération de l'algorithme des k-moyennes.

Question 11 : Tester votre programme sur l'un des tableaux proposés. Combien y-a-t il d'itérations avant stabilisation ? À partir de combien d'itérations les variations deviennent-elles négligeables ?